MAP REDUCE

Instead of bringing data to compute, why not take compute to data

↳ Paper by google (Jeff Dean & Sanjay Ghemawat)

↳ Usecase at google: computation on huge amount of data
eg: creating index of all web content
: page ranking

↳ Allows running giant computations on 1000's of computers
↳ It is a framework / paradigm that allows engineers to write the guts of the application w/o worrying about the distributed systems

↗ of GFS & MR architecture

- Hadoop is the adoption of open-source implementation by Yahoo (now Apache project)
- MapReduce is the central processing component of the Hadoop framework. It provides a programming model for processing vast amounts of data in a distributed environment, offering several key advantages:
  - **Parallel Processing:** MapReduce enables the parallel processing of data across multiple servers in a distributed computational environment, which significantly boosts performance.
  - **Data Locality:** This is a fundamental principle of MapReduce. Instead of moving large amounts of data to the processing unit, the framework moves the processing logic to where the data is already stored. This reduces expensive network traffic and improves efficiency.
- Its an execution framework for large-scale data processing
  - Which distributes the computing across distributed commodity servers and then pulls the results together
  - Programmer writes code as if writing for a single machine but the framework executes the process in distributed manner
  - Distributed implementation that hides all the messy details
    - Fault tolerance (thus provides High Availibility)
    - I/O scheduling
    - parallelization

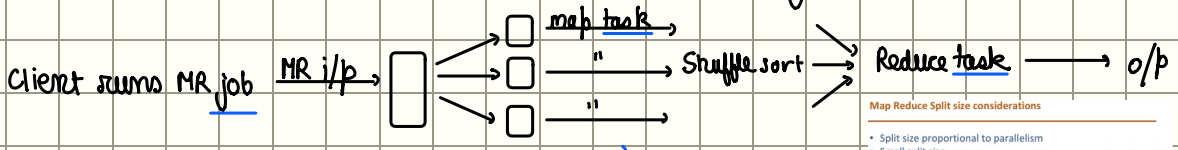
- Uses Divide and conquer – Partitions large problem into smaller subproblems
- Workers work on sub-problems in parallel (could be threads in a core or cores in multi-core processor, multiple processor in a machine, machines in a cluster) and produce intermediate results
- Intermediate results from workers are combined to form the final result

Terminology: MapReduce job - unit of work that client wants performed.

↓

divided into tasks - map tasks
- reduce tasks

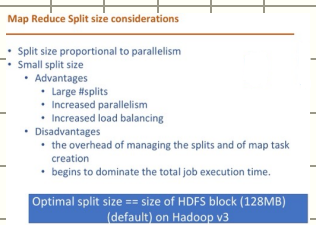
↳ scheduled using YARN ↗ automatically schedules failed tasks on another node



i/p is split

the optimal split size = chunk size in HDFS

map task is run on each record in each split



Map reduce wordcount example

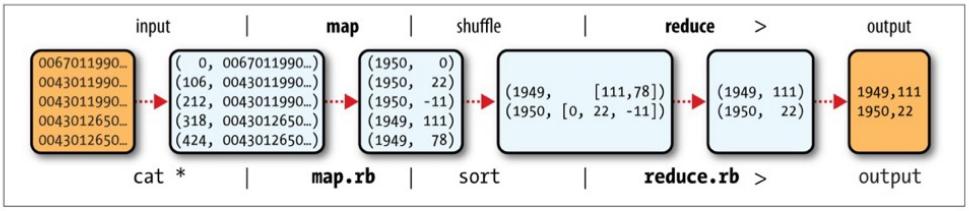


Figure 2-1. MapReduce logical data flow

file : hello world people
love hello
people hello

the i/p file is small, it will spawn just one mapper, but lets say it spawns one mapper per line

i/p to mapper : <key, value>
↳ line
↳ line num
} line num & line are just an example, it could be diff depending on our need

↓ mapper task enjoys data locality

: 0, hello world people → mapper 1
1, love hello → mapper 2
2, people hello hello → mapper 3

o/p of mappers : mapper 1 : <hello, 1>
<world, 1>
<people, 1>

↓ stored to local disk : it can be thrown away after Reducer task so storing it in HDFS w replication is overkill

2 : <love, 1>
<hello, 1>

3 : <people, 1>
<hello, 1>

all these outputs go to shuffle sort

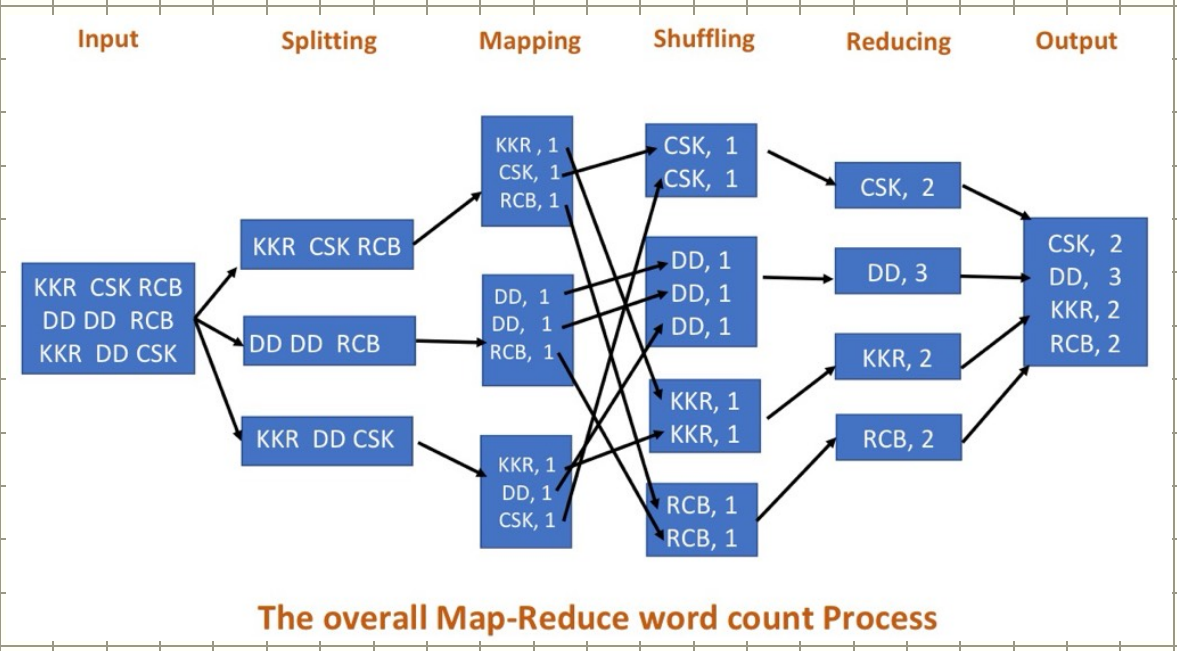
i/p of shuffle sort :
<hello, 1>
<world, 1>
<people, 1>
<love, 1>
<hello, 1>
<people, 1>
<hello, 1>

o/p of shuffle sort : hello → [1,1,1,1]
world → [1]
people → [1,1]
love → [1]

o/p of SS is i/p to reducer : reducer doesn't have advantage of data locality. sorted mapper outputs have to be transferred across network to the node where reduce task is running

o/p of reducer : hello 4
world 1
people 2
love 1

(Stored in HDFS : one replica on local node, other on off rack nodes.



The overall Map-Reduce word count Process

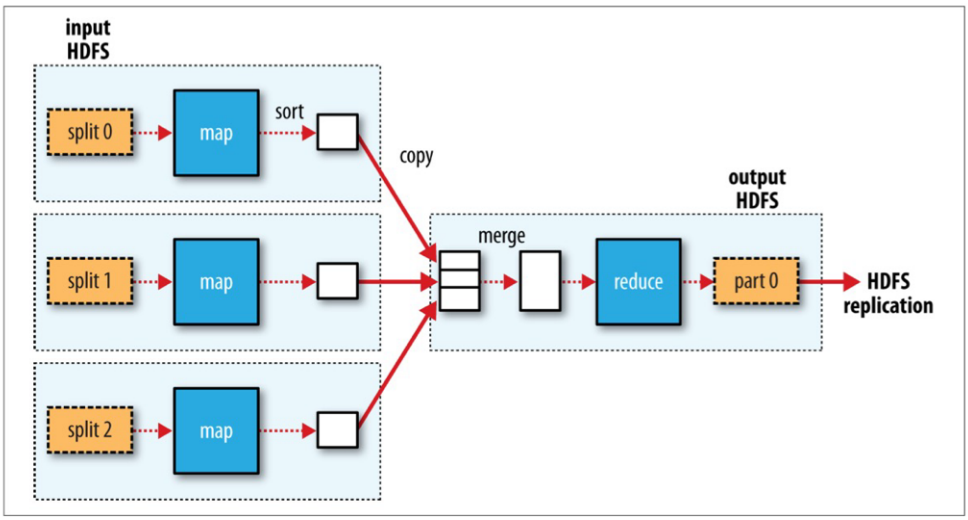


Figure 2-3. MapReduce data flow with a single reduce task

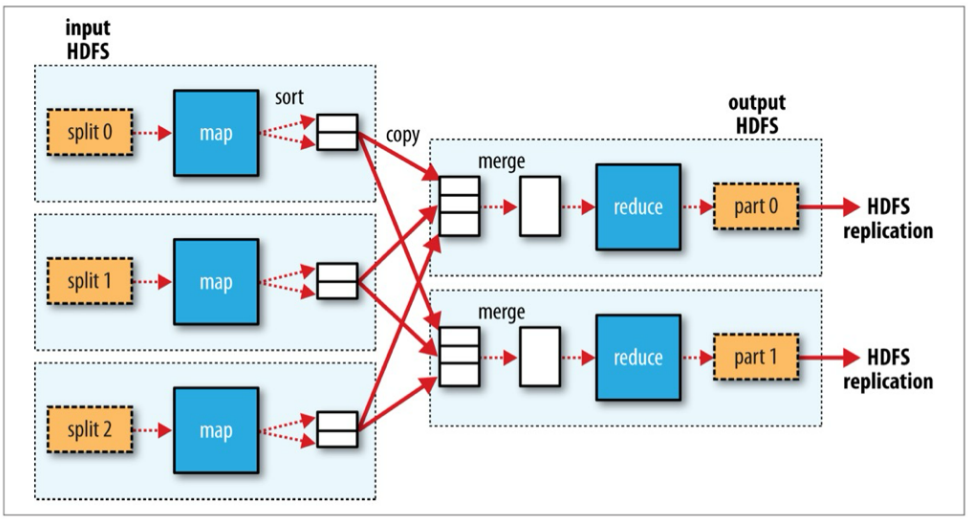


Figure 2-4. MapReduce data flow with multiple reduce tasks

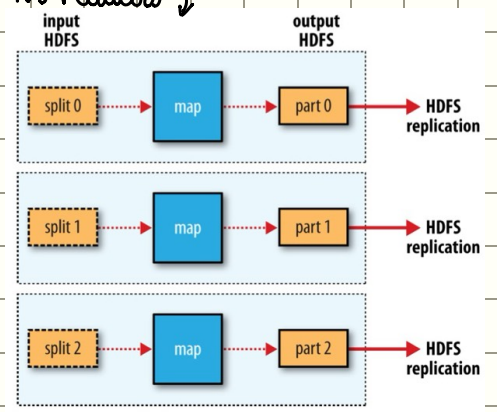
# of reduce tasks is not governed by size of i/p, it is specified independently

When there are multiple reducers, the map tasks *partition* their output, each creating one partition for each reduce task. There can be many keys (and their associated values) in each partition, but the records for any given key are all in a single partition. The partitioning can be controlled by a user-defined partitioning function, but normally the default partitioner—which buckets keys using a hash function—works very well.

Finally, it's also possible to have zero reduce tasks. This can be appropriate when you don't need the shuffle because the processing can be carried out entirely in parallel

whenever the problem does not require grouping, aggregating, or combining across multiple records, a reducer is unnecessary.

No reducers



Eg : Log file filtering :

- **Task Goal**: Extract only the error messages (lines containing "ERROR") from large log files.
- **Why reducer is not needed**: Each log line can be independently checked by the mappers. We don't need to aggregate or combine results — just output matching lines.

Data Flow

1. **Input**: A set of log files distributed across HDFS. Example:

Copy

```
2025-09-14 INFO  Server started
2025-09-14 ERROR Connection failed
2025-09-14 INFO  Request received
2025-09-14 ERROR Disk full
```

2. **Mapper Function**:
   - Reads each line.
   - Checks if the line contains "ERROR".
   - If yes, outputs the line.
   - If not, outputs nothing.

Example pseudocode:

python Copy

```
def map(key, value):
    # key: byte offset of line
    # value: log line
    if "ERROR" in value:
        emit(None, value)
```

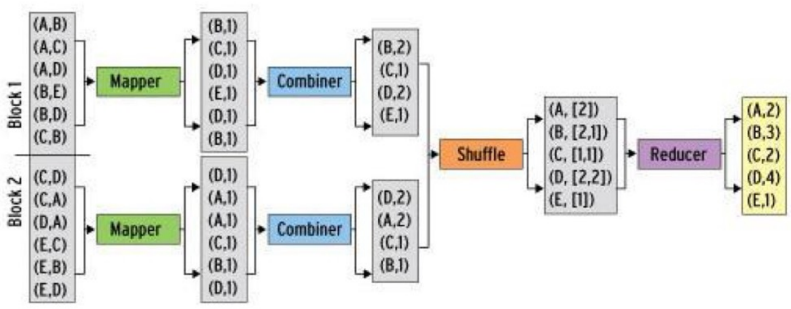
3. **Reducer**:
   - **Not needed**.
   - The framework can directly write mapper output to HDFS.
4. **Output**:

Copy

```
2025-09-14 ERROR Connection failed
2025-09-14 ERROR Disk full
```

Combiner function :

Many MapReduce jobs are limited by the bandwidth available on the cluster, so it pays to minimize the data transferred between map and reduce tasks. Hadoop allows the user to specify a *combiner function* to be run on the map output, and the combiner function's output forms the input to the reduce function. Because the combiner function is an optimization, Hadoop does not provide a guarantee of how many times it will call it for a particular map output record, if at all. In other words, calling the combiner function zero, one, or many times should produce the same output from the reducer.



- Combine multiple map outputs before doing a reduce
- Can write a combiner function in program
  - Combiner will be run before reduce
- Mini-reducer

Example : for wordcount example, there was just one occurance of each word in a line

If the i/p was : file : hello world people people people
love hello love love
people hello hello

w/o combiner : all this would have to be transferred to reducer

o/p of mappers : mapper 1 : <hello, 1>
<world, 1>
<people, 1>
<people, 1>
<people, 1>
2 : <love, 1>
<hello, 1>
<love, 1>
<love, 1>
3 : <people, 1>
<hello, 1>
<hello, 1>

w combiner

o/p of mappers : mapper 1 : <hello, 1>
<world, 1>
<people, 3>
2 : <love, 3>
<hello, 1>
3 : <people, 1>
<hello, 2>

} transferring this uses less bandwidth

Example :

This is best illustrated with an example. Suppose that for the maximum temperature example, readings for the year 1950 were processed by two maps (because they were in different splits). Imagine the first map produced the output:

```
(1950, 0)
(1950, 20)
(1950, 10)
```

and the second produced:

```
(1950, 25)
(1950, 15)
```

The reduce function would be called with a list of all the values:

```
(1950, [0, 20, 10, 25, 15])
```

with output:

```
(1950, 25)
```

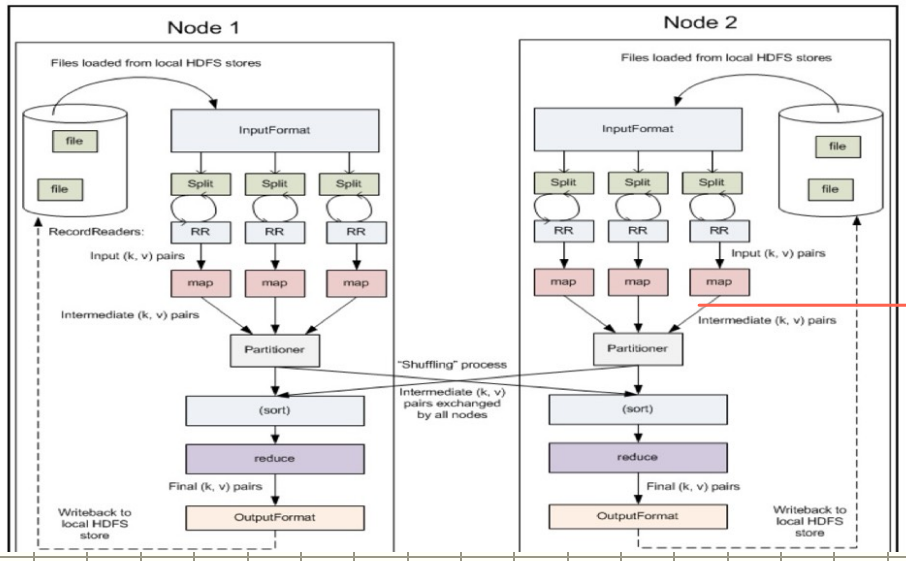
since 25 is the maximum value in the list. We could use a combiner function that, just like the reduce function, finds the maximum temperature for each map output. The reduce function would then be called with:

```
(1950, [20, 25])
```

Combiner func is usually the same as reduce function

the combiner function is defined using the Reducer class, and for this application, it is the same implementation as the reduce function in MaxTemperatureReducer. The only change we need to make is to set the combiner class on the Job

High level view



combiner comes after map & before partitioner

Hadoop Streaming

Hadoop provides an API to MapReduce that allows you to write your map and reduce functions in languages other than Java. *Hadoop Streaming* uses Unix standard streams as the interface between Hadoop and your program, so you can use any language that can read standard input and write to standard output to write your MapReduce program.[3]

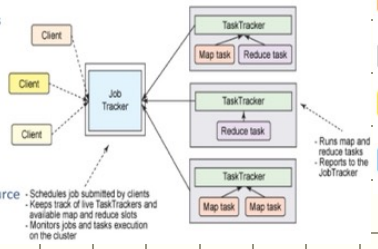
Streaming is naturally suited for text processing. Map input data is passed over standard input to your map function, which processes it line by line and writes lines to standard output. A map output key-value pair is written as a single tab-delimited line. Input to the reduce function is in the same format—a tab-separated key-value pair—passed over standard input. The reduce function reads lines from standard input, which the framework guarantees are sorted by key, and writes its results to standard output.

YARN : yet another resource negotiator

In hadoop 1

- Recall
  - Job – the entire map reduce application
  - Task – Individual mappers/reducers
- How do we
  - Allocate resources – determine which nodes will run the jobs
  - Monitor the tasks – start new tasks or restart failed/slow tasks
  - Monitor the overall state of the job?

- Job Tracker
  - Manage Cluster resources
  - Job scheduling
- Task Tracker
  - One per task
  - Manage the task
- Fault Tolerance, Cluster resource management and scheduling handled by JobTracker



Limits scalability
- Job tracker runs on a single machine and is responsible for cluster management, scheduling and monitoring

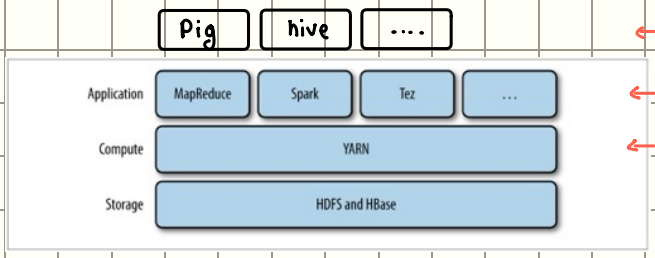
Availability
- JobTracker is the single point of availability/failure

Resource utilization problems
- Predefined #map/reduce slots. Utilization issues because map slots may be full but reduce slots are free.

Limitation in running MR applications
- Tightly integrated with Hadoop. Only MR apps can run. Can't coexist with other applications.

YARN

- Hadoop's cluster resource management system
- Provides API to applications, users don't usually directly work with yarn.



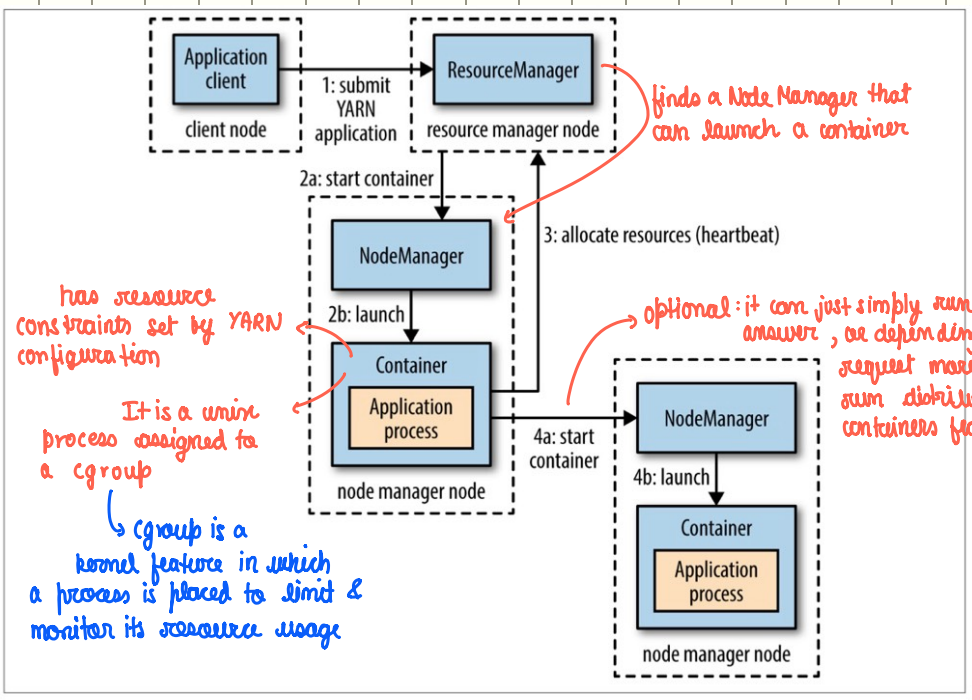
← more applications built on MR, Spark etc...
← users use
← yarn manages resources

Anatomy of YARN application run

YARN runs 2 daemons

1] Resource Manager (one per cluster) : manages resources across the cluster

2] Node Managers (runs on all nodes) : launches & monitors containers



has resource constraints set by YARN configuration

It is a unix process assigned to a cgroup
↳ cgroup is a kernel feature in which a process is placed to limit & monitor its resource usage

optional: it can just simply run computation on one node & return answer, or depending on the application (like MR), it can request more containers from resource manager to run distributed computation. Negotiating more containers from AM is task of application master

In YARN, the ApplicationMaster (AM) is a single special container that coordinates one application (e.g., a MapReduce job, a Spark job, a Tez DAG). Here's how it works:

1. Where the ApplicationMaster runs
- When you submit a YARN application:
  - The ResourceManager (RM) allocates one container for the AM.
  - This container runs the AM process.
- So, there is only one AM per application (unless it crashes and gets restarted).

2. Who tracks the status of the application
- The AM is responsible for tracking the progress of tasks running in other containers:
  - The AM requests containers from the RM.
  - The AM launches tasks in those containers (via the NodeManagers).
  - The tasks report back to the AM with status (success/failure, progress updates, heartbeat, etc.).

So the AM is the central coordinator for its application.

Figure 4-2. How YARN runs an application

Yarn resource requests

- Has flexible model for making resource requests from Resource Manager

- Application master requests RM to allocate a set of containers & can specify 2 things
  1) Computer resources (cpu, mem) for each container
  2) Locality constraints for the containers

Locality constraints help – ensure the containers use cluster bandwidth efficiently
→ would prefer not all containers on the same node to exploit full bandwidth of each node
– ensure containers are allocated close to the data they want to process → data locality also kept in mind

→ The request sent by AM to RM contains
1) how much resources for each container,
2) no of containers
3) locality for each container (hostname / rack name)
4) priority of request

locality can be requested on node basis or rack basis
First try allocating on requested node (it could be a node holding a replica of the data we need), if cannot, try in same rack & if cannot then relax constraints & allocate somewhere off rack to avoid starvation.

- The model is flexible, resource requests can be made any time an application is running. AM can request all upfront & also dynamically request.

Yarn application life span

YARN (Yet Another Resource Negotiator) applications can run for varied durations, from a few seconds to several months. A more practical way to understand them is by categorizing them based on how they correspond to user jobs. There are three primary models.

Model 1: One Application per Job

This is the simplest and most direct approach. A brand new, dedicated application is launched for every single job a user submits. Once the job is finished, the application terminates.

- **Analogy:** Think of it like hailing a **new taxi for every single trip** you make. 🚕
- **Key Characteristic: Simplicity and isolation**. Each job runs in its own environment.
- **Drawback:** It can be inefficient, as there is an overhead cost associated with starting a new Application Master for each job.
- **Example: MapReduce** follows this model.

Model 2: One Application per Workflow or Session

In this model, a single application is created to handle a complete workflow or a user's entire session, which may consist of multiple, potentially unrelated, jobs. The application remains active for the duration of the session.

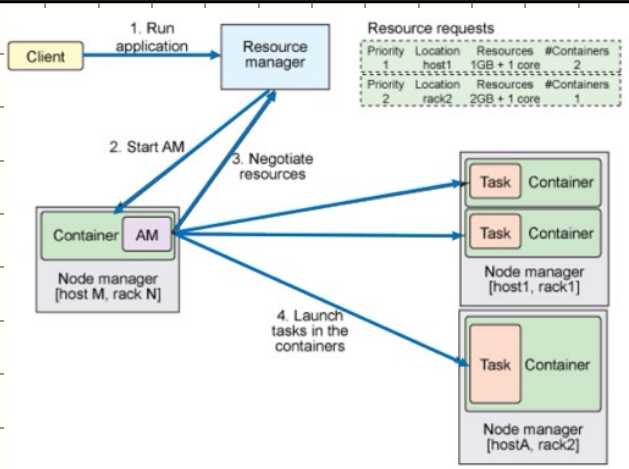
- **Analogy:** This is like **hiring a car for the entire day** to run all your errands. The car (the application) waits for you between your stops (the jobs). 🚗
- **Key Characteristic: Efficiency**. Containers can be reused across different jobs, and intermediate data can be cached, which speeds up the overall process.
- **Example: Spark** uses this model, creating an application for an interactive user session.

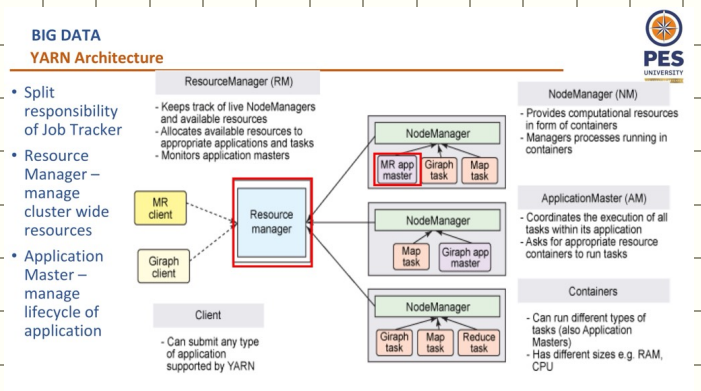
Model 3: Shared, Long-Running Application

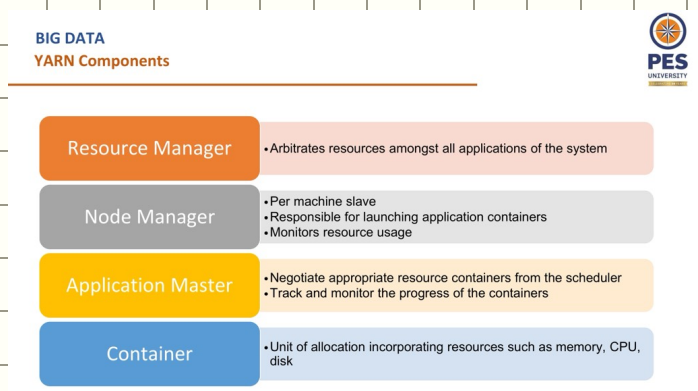
This model involves an "always-on" application that runs continuously on the cluster and is shared by many different users. It typically serves as a coordinator or a proxy for user requests.

- **Analogy:** This is similar to a **public bus service**. It's always running on its route, and many different passengers (users) can hop on to make their requests (queries) at any time. 🚌
- **Key Characteristic: Very low latency**. Since the application is already running, users get near-instant responses because the time-consuming step of starting an Application Master is completely avoided.
- **Examples:**
  - **Apache Slider**: Launches other applications on the cluster.
  - **Impala**: Uses a proxy application for its daemons to request cluster resources quickly.

Overview :







Scheduling in YARN

- Ideally resource request would be granted immediately.
- But since resources are limited, YARN will queue them they can be fulfilled.

YARN gives us 3 schedulers
1] FIFO
2] Capacity scheduler
3] Fair scheduler

1] FIFO very simple, no configuration needed

↳ Application 1 sends requests → queue → All these requests served first
↳ Application 2 sends requests → queue

⇓

#Issue with FIFO
⇒ If App1 large, all its requests will be served first, even if App2 very small, it'll have to wait long

⇒ On shared clusters, its better to use capacity or fair schedulers.

The FIFO Scheduler has the merit of being simple to understand and not needing any configuration, but it's not suitable for shared clusters. Large applications will use all the resources in a cluster, so each application has to wait its turn. On a shared cluster it is better to use the Capacity Scheduler or the Fair Scheduler. Both of these allow long-running jobs to complete in a timely manner, while still allowing users who are running concurrent smaller ad hoc queries to get results back in a reasonable time.

2] Capacity scheduler has a seperate dedicated queue ⇒ small jobs go here & start as soon as they are submitted
↳ one or more

Extra queue does have little overhead ∵ some capacity is kept away for small jobs
⇒ long tasks take a tad bit longer than FIFO

Each queue internally is FIFO scheduled

Issue : if one queue is not being used, no other job can use the extra unused resources

3] Fair Scheduler dynamically balances resources b/w all running jobs.

If only one job present ⇒ it uses 100% resources, if second comes, it is allocated 1/2 resources
⇒ fair sharing

there is lil bit lag before 2nd job starts ∵ first job is freeing up some space for it.

- Consider a user who submits more jobs
  - Scheduler ensures that user does not hog the cluster
- Custom pools
  - Guaranteed minimum capacities with map/reduce slots
  - It is also possible to define custom pools with guaranteed minimum capacities defined in terms of the number of map
- The Fair Scheduler supports preemption
  - If pool not received its fair share over certain time
  - scheduler will kill tasks in pools running over capacity

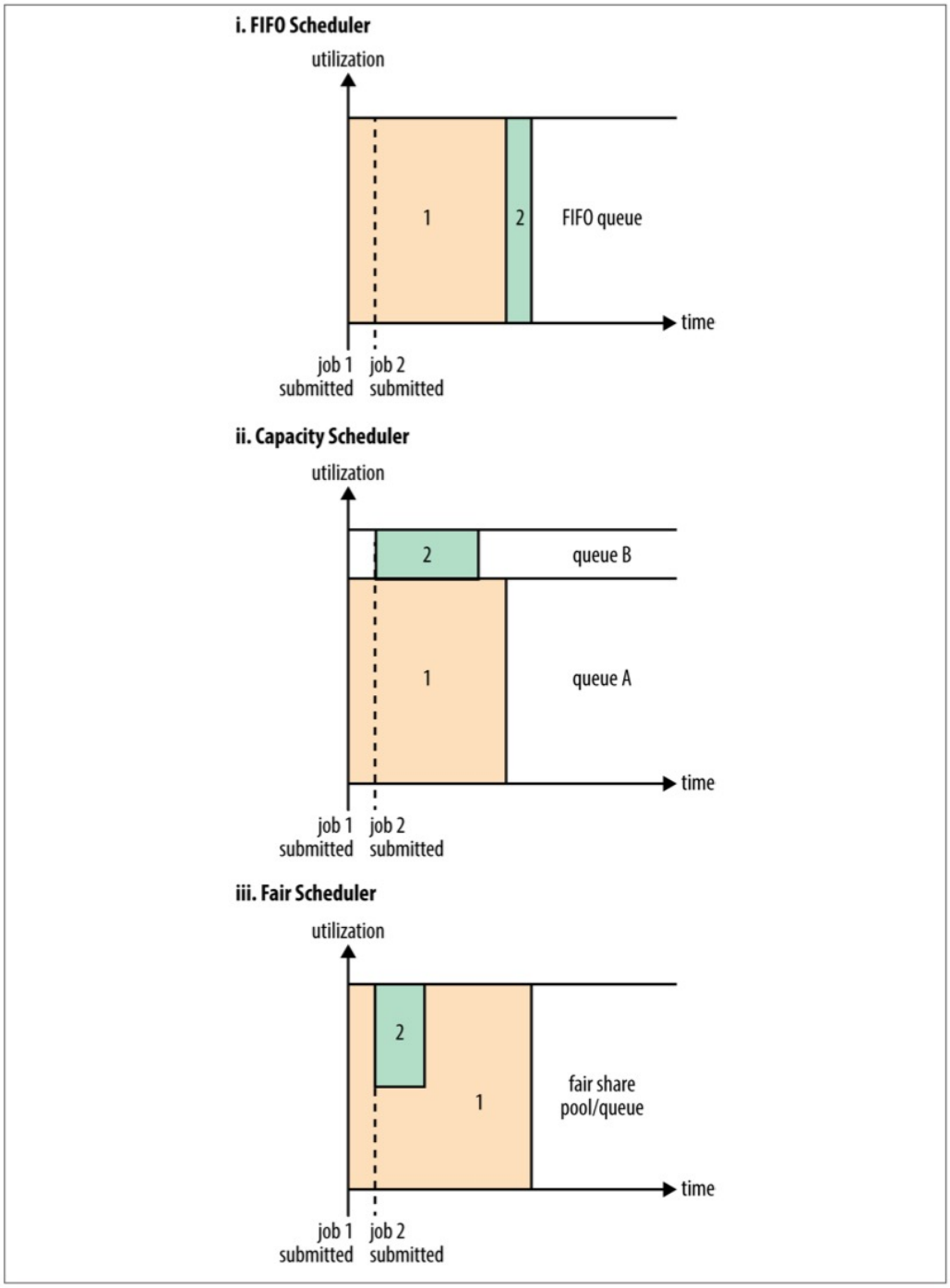
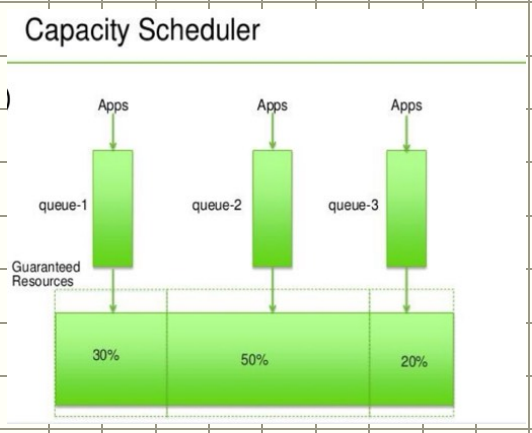


Figure 4-3. Cluster utilization over time when running a large job and a small job under the FIFO Scheduler (i), Capacity Scheduler (ii), and Fair Scheduler (iii)



Fair Scheduler